

Universidad San Carlos de Guatemala  
Facultad de ingeniería.  
Ingeniería en ciencias y sistemas



**FIUSAC**  
FACULTAD DE INGENIERÍA  
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



## Práctica:

GitOps, entrega progresiva y seguridad  
de la cadena de suministro

**PONDERACIÓN: 3.75 pts**

 **Tiempo estimado: 14 hrs**

## Índice

### Contenido

|                                                  |    |
|--------------------------------------------------|----|
| 1. MARCO FORMATIVO.....                          | 4  |
| 1.1. Valor.....                                  | 4  |
| 1.2. Competencia(s).....                         | 4  |
| 1.3. Habilidad(es) blandas a formar.....         | 4  |
| 2. Resultado del Aprendizaje .....               | 5  |
| 2.1. Objetivo SMART.....                         | 5  |
| 3. Enunciado de la Práctica.....                 | 6  |
| 3.1 Descripción del problema a resolver.....     | 6  |
| 3.2 Alcance de la práctica.....                  | 6  |
| 3.3 Requerimientos técnicos .....                | 8  |
| 4. Entregables .....                             | 9  |
| 4.1 Tabla de enlaces obligatoria.....            | 10 |
| 4.2 Plantilla del informe de incidente .....     | 11 |
| 5. Material de apoyo.....                        | 12 |
| 6. Recursos y herramientas a utilizar .....      | 12 |
| 7. Cronograma .....                              | 13 |
| 8. Rúbrica de Calificación.....                  | 13 |
| 8.1 Requisitos para optar a la calificación..... | 13 |
| 8.2 Resumen de Puntuaciones.....                 | 14 |
| Detalle de la Calificación .....                 | 16 |

Comentarios Generales..... ¡Error! Marcador no definido.

# 1. MARCO FORMATIVO

## 1.1. Valor

| Nombre del valor                                      | ¿Cómo se aplica en tu laboratorio?                                                                                                                                                                                                                                          |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Integridad y trazabilidad en la operación de software | El estudiante deberá garantizar que todo cambio aplicado a un entorno productivo quede registrado, revisado y sea reversible, evitando modificaciones manuales no trazables sobre la infraestructura y asegurando que únicamente artefactos verificados lleguen al clúster. |

## 1.2. Competencia(s)

Con la elaboración de esta práctica usted adquirirá las siguientes competencias:

- **Competencia General:** Operar sistemas distribuidos bajo un modelo declarativo en el que el repositorio es la única fuente de verdad, aplicando controles automáticos de calidad y seguridad antes de exponer un cambio a los usuarios.
- **Competencia Específica:** Diseñar e implementar un flujo GitOps con entrega progresiva, validación automatizada, políticas de admisión y verificación de artefactos para un ecosistema de microservicios desplegado en Kubernetes.

## 1.3. Habilidad(es) blandas a formar

La práctica le permitirá desarrollar las siguientes habilidades:

- **Pensamiento crítico ante el fallo:** Anticipar el comportamiento del sistema cuando un despliegue sale mal y diseñar mecanismos que lo contengan sin intervención humana.

- **Rigurosidad operativa:** Respetar procesos y controles automáticos aun cuando exista una vía manual más rápida para lograr el mismo resultado.
- **Comunicación técnica efectiva:** Documentar un incidente y su resolución de forma que un tercero pueda comprender la causa, la detección y la prevención.

## 2. Resultado del Aprendizaje

### 2.1. Objetivo SMART

| Específico<br>(¿Qué?)                                                                                                                          | Medible<br>(¿Cuánto?)                                                                                                                           | Alcanzable<br>(¿Cómo?)                                                                                                                  | Realista (¿Para<br>qué?)                                                                                            | A Tiempo<br>(¿Cuándo?)                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Implementar un flujo GitOps con entrega progresiva y validación automatizada para el ecosistema de microservicios de las prácticas anteriores. | Demostrar 1 promoción exitosa de versión, 1 reversión automática ante una versión defectuosa y 1 despliegue rechazado por política del clúster. | Utilizando ArgoCD, Argo Rollouts, Helm, Terraform, políticas de admisión y verificación de imágenes sobre el pipeline de la Práctica 7. | Adquirir criterio para operar entregas de software donde un despliegue erróneo impacta directamente a los usuarios. | Antes del XX de noviembre de 2026 a las 23:59 hrs, entregando el repositorio vía UEDI. |

### 3. Enunciado de la Práctica.

#### 3.1 Descripción del problema a resolver

Luego de haber automatizado la construcción y el despliegue del sistema de microservicios, el flujo de la Práctica 7 presenta tres debilidades que ninguna organización acepta en producción:

- El despliegue avanza a ciegas: un commit defectuoso alcanza al 100 % de los usuarios sin validación intermedia.
- El pipeline posee credenciales de administración del clúster, por lo que comprometer el repositorio implica comprometer la infraestructura.
- El estado real del clúster puede diferir del estado declarado en el repositorio y nadie lo detecta.

El estudiante deberá evolucionar el flujo hacia un modelo GitOps, en el que el repositorio es la única fuente de verdad, ningún cambio avanza sin superar validaciones automáticas y toda versión defectuosa es revertida por el propio sistema. Adicionalmente, ninguna imagen podrá llegar al clúster sin haber sido analizada, firmada y verificada.

#### 3.2 Alcance de la práctica

**El estudiante deberá cumplir con las siguientes actividades dentro del flujo de entrega:**

- **Infraestructura como código:** namespaces, cuotas de recursos, límites y RBAC del clúster definidos y aplicados con Terraform. No se acepta infraestructura creada manualmente.
- **Empaquetado con Helm:** un chart por microservicio, con archivos de valores diferenciados por ambiente y validación mediante helm lint dentro del pipeline.
- **GitOps con ArgoCD:** repositorio de manifiestos independiente del repositorio de código. El pipeline únicamente actualiza la versión de la imagen mediante un Pull Request automático; ArgoCD es el único componente que aplica cambios al clúster.
- **Prohibición de despliegue directo:** los workflows no podrán contener kubeconfig ni ejecutar kubectl apply, kubectl set image o helm upgrade contra el clúster.

- **Entrega progresiva con Argo Rollouts:** estrategia canary o blue-green con al menos tres pasos de promoción, cada uno condicionado a un AnalysisTemplate que ejecute pruebas contra la nueva versión.
- **Validación automatizada como puerta de calidad:** pruebas de humo e integración sobre los endpoints críticos y prueba de carga con k6 o Locust. El estudiante definirá y justificará los umbrales de error y de tiempo de respuesta que determinan la promoción o la reversión.
- **Seguridad de la cadena de suministro:** análisis con Trivy que bloquee el Pull Request ante CVE críticas, generación de SBOM, firma de imágenes con Cosign y verificación de la firma antes del despliegue.
- **Políticas como código:** Kyverno u OPA Gatekeeper con al menos tres políticas obligatorias: prohibir la etiqueta latest, exigir límites de CPU y memoria, y exigir ejecución sin privilegios de root.
- **Gestión de secretos:** Sealed Secrets o External Secrets. No se admite ningún secreto en texto plano en el repositorio de manifiestos.
- **Versionamiento semántico:** la etiqueta de la imagen deberá derivarse de un tag o release de Git. Queda prohibido el uso de latest.
- **Fallo inducido:** publicar deliberadamente una versión defectuosa y demostrar con evidencia que las validaciones del canary la detectan y el sistema revierte automáticamente.
- **Uso del sistema de microservicios previo:** Prácticas 5, 6 y 7.

### 3.3 Requerimientos técnicos

El estudiante deberá utilizar o integrar las siguientes herramientas y tecnologías:

- Repositorio GitHub de código y repositorio GitHub independiente de manifiestos.
- GitHub Actions y pipelines en YAML.
- Terraform.
- Helm.
- ArgoCD y Argo Rollouts.
- Kyverno u OPA Gatekeeper.
- Trivy y Cosign.
- k6 o Locust.
- Docker y Kubernetes.

## 4. Entregables

| Tipo                       | Descripción                                                                                                                                |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Repositorio de código      | Contener carpeta /P8 con workflows, charts, pruebas y documentación.                                                                       |
| Repositorio GitOps         | Repositorio público independiente con los manifiestos declarativos del sistema.                                                            |
| README de entrega          | Ubicado en /P8, con la tabla de enlaces obligatoria de la sección 4.1. Es el único documento que se utilizará para localizar la evidencia. |
| Código Terraform           | Namespaces, cuotas, límites y RBAC, con evidencia de plan y apply.                                                                         |
| Helm charts                | Un chart por servicio, con valores diferenciados por ambiente.                                                                             |
| Evidencia de ArgoCD        | Aplicaciones en estado Synced y Healthy, e historial de sincronizaciones.                                                                  |
| Evidencia de Argo Rollouts | Capturas o registros de la promoción por pasos y de la reversión automática.                                                               |
| Pruebas automatizadas      | Scripts de humo, integración y carga, con sus reportes de ejecución.                                                                       |

| Tipo                   | Descripción                                                                                    |
|------------------------|------------------------------------------------------------------------------------------------|
| Evidencia de seguridad | Reporte de Trivy, archivo SBOM y salida de verificación de firma con Cosign.                   |
| Políticas de admisión  | Manifiestos de las políticas y evidencia de un despliegue rechazado.                           |
| Informe de incidente   | Análisis del fallo inducido según la plantilla de la sección 4.2.                              |
| Diagrama del flujo     | Arquitectura visual desde el commit hasta el clúster, con puntos de validación y de reversión. |
| Video demostrativo     | De 5 a 8 minutos, con minutaje indicado en el README.                                          |

#### 4.1 Tabla de enlaces obligatoria

El README de la carpeta /P8 deberá incluir la siguiente tabla completamente llena. Un enlace ausente, roto o que requiera autenticación se calificará con cero en el criterio correspondiente, sin búsqueda adicional dentro del repositorio.

| Ítem                 | Enlace o dato requerido                    |
|----------------------|--------------------------------------------|
| Repositorio GitOps   | URL pública                                |
| Aplicación en ArgoCD | Nombre exacto de la aplicación y namespace |

| Ítem                               | Enlace o dato requerido                      |
|------------------------------------|----------------------------------------------|
| Ejecución exitosa del pipeline     | URL directa al run de GitHub Actions         |
| Reversión automática               | URL del run y del Rollout donde se evidencia |
| Despliegue rechazado por política  | URL directa a la evidencia del rechazo       |
| Bloqueo por vulnerabilidad crítica | URL directa al Pull Request bloqueado        |
| Imagen firmada                     | Referencia completa registro/imagen:tag      |
| Reporte de prueba de carga         | Ruta del archivo dentro del repositorio      |
| Video demostrativo                 | URL y minutaje por punto demostrado          |

## 4.2 Plantilla del informe de incidente

El informe del fallo inducido no deberá exceder una página y deberá responder exactamente estos cinco campos:

- Qué falló: naturaleza del defecto introducido deliberadamente.
- Cómo se detectó: validación específica que lo identificó y umbral superado.
- Cómo se contuvo: mecanismo que ejecutó la reversión y porcentaje de tráfico afectado.

- Tiempo de recuperación: minutos entre la publicación y el retorno al estado estable.
- Cómo prevenirlo: control adicional que habría evitado que la versión defectuosa llegara al canary.

## 5. Material de apoyo

- Documentación oficial de ArgoCD: <https://argo-cd.readthedocs.io>
- Documentación oficial de Argo Rollouts: <https://argo-rollouts.readthedocs.io>
- Documentación oficial de Helm: <https://helm.sh/docs/>
- Documentación oficial de Terraform: <https://developer.hashicorp.com/terraform/docs>
- Documentación oficial de Kyverno: <https://kyverno.io/docs/>
- Documentación de Trivy y Cosign (proyecto Sigstore).
- Documentación oficial de k6: <https://grafana.com/docs/k6/>
- Principios GitOps de la OpenGitOps Working Group.

## 6. Recursos y herramientas a utilizar

- **Versionamiento y automatización:** GitHub, GitHub Actions.
- **Infraestructura como código:** Terraform.
- **Empaquetado y despliegue:** Helm, ArgoCD, Argo Rollouts.
- **Contenedores y registro:** Docker, DockerHub o GHCR.
- **Orquestación:** K8s (Kubernetes).

- **Seguridad:** Trivy, Cosign, Kyverno u OPA Gatekeeper, Sealed Secrets o External Secrets.
- **Pruebas:** k6 o Locust y el framework de pruebas del lenguaje utilizado.

## 7. Cronograma

| Tipo                   | Fecha Inicio | Fecha Fin  |
|------------------------|--------------|------------|
| Asignación de Práctica | 10/09/2026   | 10/09/2026 |
| Elaboración            | 10/09/2026   | 17/09/2026 |
| Calificación           | 19/09/2026   | 19/09/2026 |

## 8. Rúbrica de Calificación

### 8.1 Requisitos para optar a la calificación

Antes de la evaluación de la práctica, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

| Tema             | Descripción                                                                          | Cumple (Sí/No) |
|------------------|--------------------------------------------------------------------------------------|----------------|
| Práctica 7       | Contar con el desarrollo de la Práctica 7 previamente calificado.                    |                |
| Estado de ArgoCD | La aplicación se encuentra en estado Synced y Healthy al momento de la calificación. |                |

| Tema                   | Descripción                                                                                                        | Cumple (Sí/No) |
|------------------------|--------------------------------------------------------------------------------------------------------------------|----------------|
| Sin despliegue directo | Ningún workflow ejecuta kubectl apply, kubectl set image o helm upgrade contra el clúster, ni almacena kubeconfig. |                |
| Evidencia de reversión | Existe evidencia verificable de al menos una reversión automática provocada por una validación fallida.            |                |
| README de entrega      | La tabla de enlaces obligatoria está completa y todos los enlaces son accesibles públicamente.                     |                |

## 8.2 Resumen de Puntuaciones

| Área                        | Puntos Totales (Base 100) | Puntos Obtenidos |
|-----------------------------|---------------------------|------------------|
| <b>1. Habilidades (40%)</b> |                           |                  |
| Documentación técnica       | 8                         |                  |
| Diagrama del flujo GitOps   | 8                         |                  |
| Informe de incidente        | 10                        |                  |

| Área                                      | Puntos Totales<br>(Base 100) | Puntos<br>Obtenidos |
|-------------------------------------------|------------------------------|---------------------|
| Preguntas teóricas                        | 14                           |                     |
| <b>Sub-Total Habilidades</b>              | <b>40</b>                    |                     |
| <b>2. Conocimiento (60%)</b>              |                              |                     |
| Infraestructura como código (Terraform)   | 8                            |                     |
| Empaquetado con Helm                      | 8                            |                     |
| GitOps con ArgoCD                         | 14                           |                     |
| Entrega progresiva y reversión automática | 14                           |                     |
| Validación automatizada                   | 8                            |                     |
| Cadena de suministro y políticas          | 8                            |                     |
| <b>Sub-Total Conocimiento</b>             | <b>60</b>                    |                     |
| <b>TOTAL (Escalable a 3.75 pts)</b>       | <b>100</b>                   |                     |

### Detalle de la Calificación

| No.      | Criterio de evaluación    | Punteo máximo | Satisfactorio (100% - 61%)                                                                                                  | Necesita mejorar (60% - 0%)                                                        | Punteo Obtenido |
|----------|---------------------------|---------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-----------------|
| <b>1</b> | <b>Habilidades</b>        | <b>40</b>     |                                                                                                                             |                                                                                    |                 |
| 1.1      | Documentación técnica     | 8             | Explica con claridad el flujo GitOps, los puntos de validación y las decisiones de diseño, en un máximo de dos páginas.     | Explicación genérica o incompleta sobre el flujo implementado.                     |                 |
| 1.2      | Diagrama del flujo GitOps | 8             | Muestra el recorrido del commit al clúster, distinguiendo qué componente aplica los cambios y dónde ocurre cada validación. | Diagrama confuso, ausente o que no refleja la implementación entregada.            |                 |
| 1.3      | Informe de incidente      | 10            | Responde los cinco campos de la plantilla con datos concretos y propone un control preventivo pertinente.                   | Narración vaga, sin tiempos, sin causa identificada o sin propuesta de prevención. |                 |
| 1.4      | Preguntas teóricas        | 14            | Analiza el comportamiento de su propia implementación y sus implicaciones operativas.                                       | Respuestas copiadas, contradictorias con su implementación o sin análisis.         |                 |

| No. | Criterio de evaluación         | Punteo máximo | Satisfactorio (100% - 61%)                                                                                                         | Necesita mejorar (60% - 0%)                                                       | Punteo Obtenido |
|-----|--------------------------------|---------------|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------|
| 2   | Conocimiento                   | 60            |                                                                                                                                    |                                                                                   |                 |
| 2.1 | Infraestructura como código    | 8             | Namespaces, cuotas, límites y RBAC aplicados desde Terraform, sin recursos creados manualmente.                                    | Terraform ausente, incompleto o infraestructura creada a mano.                    |                 |
| 2.2 | Empaquetado con Helm           | 8             | Charts propios, parametrizados por ambiente y validados con helm lint en el pipeline.                                              | Manifiestos estáticos duplicados, sin parametrización ni validación.              |                 |
| 2.3 | GitOps con ArgoCD              | 14            | ArgoCD es el único componente que aplica cambios; el pipeline solo genera el PR de versión y la aplicación permanece sincronizada. | Existe despliegue directo desde el pipeline o el clúster difiere del repositorio. |                 |
| 2.4 | Entrega progresiva y reversión | 14            | El rollout avanza por pasos condicionados al análisis y revierte automáticamente ante la versión defectuosa.                       | Promoción incondicional, reversión manual o rollout inexistente.                  |                 |

| No. | Criterio de evaluación           | Punteo máximo | Satisfactorio (100% - 61%)                                                                                               | Necesita mejorar (60% - 0%)                                             | Punteo Obtenido |
|-----|----------------------------------|---------------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|-----------------|
| 2.5 | Validación automatizada          | 8             | Pruebas de humo, integración y carga ejecutadas contra la versión candidata, con umbrales justificados.                  | Pruebas triviales, siempre exitosas o desligadas de la promoción.       |                 |
| 2.6 | Cadena de suministro y políticas | 8             | Trivy bloquea, la imagen está firmada y verificada, las políticas rechazan lo no conforme y los secretos están cifrados. | Controles ausentes, sin capacidad de bloqueo o secretos en texto plano. |                 |